Computer Hydrological Forecasting

Generating synthetic monthly discharge records of arbitrary length from a fitted ARIMA model, for reservoir and spillway design

Author: Ugbodaga Benedict Osikpemi (190402003)
Department: Civil and Environmental Engineering
Institution: University of Lagos
Supervisor: Prof. K. O. Aiyesimoju
Submission: February 2026

This document explains the project in three parts: first in very simple terms, then the real technical version, and finally how to actually use the software. It starts with what changed most recently, since that's what makes everything after it correct.

0. What Changed Most Recently (and Why)

The output changed from a forecast to a synthetic record (2026-08-19)

The supervisor reviewed the web app by phone on 2026-08-19 and rejected it. The app at the time asked for a target date range and showed a value for a chosen month. His objection, in his own words: "That's no use to us! It's the whole range!" What he wants the model to produce is a long table of monthly values -- "you want it to be next 30 years, it will produce it; you want it to be next 1,000 years, it will produce it" -- because the engineering purpose is to read the extremes off a long record and size a reservoir or spillway against them. There is no such thing as the discharge of a particular future month; there is only the distribution the river can produce.

He is right, and the app now does exactly that. It asks for one number, how many years, and returns the record itself: one row per month, downloadable, with the return periods computed from it. It cannot be asked for a single named month at all, which is deliberate.

There is a second reason this was the correct change, and it is worth knowing. The old date-range control was not merely unhelpful -- it was meaningless. The fitted process is stationary, so every window of a given length has an identical distribution. Asking for 2030-2035 rather than 2050-2055 returned statistically the same thing. The app was already generating synthetic record and only the interface pretended otherwise.

The annual cycle is now removed explicitly -- and how, is the one place this project departs fro

The supervisor also said the differencing order cannot be zero for monthly data -- "monthly cannot be zero... if it's monthly data, it's ARIMA, not ARMA" -- and followed up specifying that the differencing factor should be 12, that is, X(t) - X(t-12). He is right about the substance: the annual cycle is the largest systematic feature of the record, roughly half its variance, and it must be removed. He is also right that ordinary differencing cannot remove it.

The difficulty is that his two requirements cannot both be met literally. Differencing at lag 12 leaves an integrated process: rebuilding the actual flow requires a running total, and a running total of random terms is a random walk, whose spread grows without limit. Over the 1,000-year record he asked for, the generated series drifts to an average of about 10^10 cubic metres per second against an observed average of 16.4, a factor of roughly 10^9 between its first decade and its last. A model that differences cannot generate a long record. A generating model has to be stationary.

The cycle is therefore removed the other standard way, by seasonal standardisation: the average and spread of each of the twelve calendar months are estimated, and every observation is expressed as a departure from its own month's average. This removes exactly what he wants removed, keeps the generator stationary, is the classical treatment in stochastic hydrology for precisely this purpose (Salas et al., 1980), and still puts the number twelve at the centre of the model -- as twelve pairs of parameters rather than as a differencing lag.

The lag-12 differencing model is fully implemented, fitted on every pipeline run, and reported side by side with the adopted one in Table 2 of Chapter 4 and in the app's "For the curious" panel. If the supervisor asks why his instruction was not followed literally, the answer is not an argument, it is that table: both models fit the observed record about equally well, and only one of them can produce the thousand years he asked for.

A supporting diagnostic: when a seasonal difference is applied to a cycle that repeats in a nearly fixed shape, the fitted seasonal moving-average coefficient is driven towards -1, because it has to undo most of what the differencing just did. The estimate here is about -0.88. The model is itself reporting that differencing the cycle was largely unnecessary.

The report is now discharge-only

Confirmed with the supervisor on 2026-08-19: discharge alone is the subject. Rainfall and stage remain in the codebase and appear once, in Section 4.7, where the identical unmodified program is run on all three to show that it selects a different order for each -- his own point that "the order of discharge cannot be the order of rainfall, cannot be the order of stock."

What this fixed, as a side effect

Treating the cycle explicitly resolved a defect the previous version reported honestly as a limitation. The Ljung-Box test used to reject white-noise residuals for discharge ($p = 0.0035$) and stage ($p = 0.0011$), meaning structure remained that the model had not captured. All three variables now pass. The models are also smaller than before, not larger: the improvement came from describing the seasonality properly, not from adding flexibility.

Earlier: the modelling approach pivoted (2026-08-12)

The modelling approach pivoted on 2026-08-12, on the supervisor's direct instruction, and it matters for reading everything that follows correctly.

The modelling approach pivoted

- **Monthly, not daily:** A monthly timestep is what makes the differencing order a meaningful thing to test at all -- it strips daily noise and exposes the annual cycle and any slow drift in level. It does not mean ordinary differencing removes seasonality; see the note below.

- **Three variables, not one:** Discharge, rainfall and stage are each modelled independently, to show the same ARIMA method generalises across variable types, not just discharge.

- **Stochastic validation:** A stochastic model's individual forecasts are not meant to match the one thing that actually happened; only its statistical properties -- mean, variability, seasonality, drought duration, peak size -- should match history.

An even earlier version of this project (before either of the above) used a rainfall-runoff model on a Nigerian basin; that was replaced by the ARIMA approach on the same supervisor's instruction, and is not revisited here -- see CLAUDE.md and archive/ for that history if needed.

The report was then put through two rounds of critical review

After the pivot above, the thesis text was independently reviewed twice (each round: three reviewer personas instructed to challenge why, how, and where every claim came from, not just skim the document). Real, checkable problems came out of both rounds. A table-numbering gap, a data-vs-text arithmetic mismatch, and two backwards test-result claims (ARCH and Jarque-Bera) were fixed directly. Three deeper methodological gaps -- no worked reservoir/spillway design example, an unsupported cross-basin transfer claim, and an unstated discharge/stage independence caveat -- were resolved with real supplementary analysis rather than softened wording (see 'On generalisation and design use' in Section 2). Three other requests -- make the persistence skill score the headline metric, reintroduce an explicit Duan bias-correction step, and drop the qualification on the cross-basin claim -- were considered and declined, because each would have reintroduced exactly the point-forecast framing the 2026-08-12 pivot moved away from.

A third review pass tightened seven claims (2026-08-15)

A further review, this time of the overview document and the live application rather than the thesis text, found seven statements that were defensible in spirit but imprecise, over-absolute, or mutually inconsistent as written. All seven have been corrected in the report, this document, and the app. They are listed here because each is a plausible defence question, and the corrected position is the one to give:

- **1. Differencing and seasonality:** Ordinary differencing, X(t) - X(t-1), removes trend or slow drift. It does NOT remove an annual seasonal cycle -- that needs seasonal differencing, X(t) -

X(t-12), or SARIMA terms. Monthly aggregation makes the differencing order testable and exposes the annual cycle; it does not turn ordinary differencing into a seasonal filter. In the event the tests chose d = 0 for all three variables, so these models do not difference at all, and the seasonal structure left in the residuals is exactly why SARIMA is the leading item of further work.

- **2. Residual requirement:** Do not say all three models fit well. Residuals should resemble white noise; Ljung-Box rejects that null for discharge (p = 0.0035) and stage (p = 0.0011), and does not reject it for rainfall (p = 0.9602). Rainfall has the cleanest statistical fit; discharge and stage retain unexplained temporal structure.

- **3. What 7/7 means:** The property score is not an accuracy grade. 7 of 7 means seven selected historical properties fell inside the simulated 90 per cent envelopes -- evidence of property reproduction, not point-forecast accuracy, not a percentage correct, not prediction error, and not the reliability of any individual future value. The phrase 'the closest thing this model has to an accuracy grade' has been removed from the app.

- **4. Comparison is not impossible:** Saying that comparing a stochastic forecast with observations is meaningless is too absolute. What is inappropriate is judging one random realisation as if it were a deterministic forecast. The observation can properly be scored against the full predictive distribution -- prediction-interval coverage, CRPS, the logarithmic score, calibration plots, rank histograms, Brier score for threshold events. Property-based validation is one such distribution-level comparison; those scores are complementary to it, not excluded by it.

- **5. Cross-basin consistency:** The report says the procedure was rerun on two additional basins; the app's limitations section said transfer was untested. The app now states the same qualified position as the report: the procedure transfers and ran cleanly on both extra basins, but its qualitative findings did not universally repeat, two of four extra fits showed numerical warning signs, and the full stochastic validation still covers the primary basin only.

- **6. Long-range forecasts are conditional:** The app accepts a target year as far out as 2200. ARIMA is mathematically defined there, but a year-2200 hydrological forecast is not scientifically reliable: it assumes the process estimated from 1980-2003 continues unchanged despite climate change, land-use change, reservoir construction, river engineering, urbanisation, and gauge or measurement changes. Long-range output is now presented as a synthetic scenario consistent with the basin's historical statistics -- which is what a design calculation needs -- not as a prediction.

- **7. Log back-transformation:** Exponentiating an expected log value gives approximately a median, not the arithmetic mean, so a retransformation bias correction would normally be needed. It is not needed here, and this was verified against the code: every one of the 1,000 simulated paths is exponentiated individually and all statistics are computed afterwards on the natural-scale ensemble, so Monte Carlo integration handles the log-normal skew automatically and the ensemble mean is unbiased by construction. Duan's smearing factor is computed as a diagnostic only and multiplies nothing. The single line the app draws through the band is the ensemble median, and is now labelled as such rather than as an 'expected path'.

1. Questions and Answers From Last Review

From the 2026-08-19 review (most recent -- answer these first)

Q: Why is the differencing order still zero? I told you it must be 12 for monthly data.

The differencing at lag 12 is in the program, sir, and I fitted it -- it is in Table 2 of Chapter 4 side by side with what I ended up using. The problem is that it cannot do the other thing you asked for. When you difference at lag 12 you have to add the values back up again to get the actual flow, and adding up random terms gives you a random walk, so the spread keeps growing the longer you run it. Over 1,000 years the record drifts to about 10^10 cubic metres per second when the river averages 16. So it fits the 35 years fine but it cannot generate the long record. What I did instead was take the seasonality out directly -- the average and the spread of each of the twelve months, so it is still twelve, just as twelve parameters rather than a differencing lag. That removes the same seasonality, and it stays stationary, so I can generate as many years as you want.

-> Where to look: Report Section 4.2, Table 2; app 'For the curious' panel

Q: So how do I know differencing at lag 12 is really worse? Show me.

Both of them fit the observed record about the same -- the residuals pass the Ljung-Box test either way, so on fit alone you genuinely cannot choose between them. The difference only shows up when you generate. Table 2 has both: the differenced one drifts by a factor of about 10^9 from its first decade to its last, and the one I used comes out at 0.95, essentially no drift. I should also say the two AIC values are not comparable, because they are computed on differently transformed series, so I have deliberately not put them next to each other as if they were.

-> Where to look: Report Section 4.2, Table 2; Section 3.5 on comparability

Q: Your seasonal moving-average coefficient came out at -0.88. What does that tell you?

That is the model telling me the differencing was not really needed. When you difference a cycle that repeats in almost the same shape every year, the moving-average term has to undo most of what the differencing just did, so the coefficient gets pushed towards -1. Getting -0.88 says this river's annual cycle is close to deterministic -- which is exactly the case where you should be taking the seasonality out directly instead of differencing it.

-> Where to look: Report Section 4.2, final paragraph

Q: Show me the output for 1,000 years.

Yes sir -- you type 1,000 into the box and it prints all 12,000 months, and you can download the whole table. Underneath it gives the design flow for each return period, which is what you would actually use to size something: about 102 cubic metres per second for the 10-year, 239 for the 100-year. The check I would point to is that the 10-year figure lands almost exactly on the biggest month we actually measured in 35 years, which is roughly where a 10-year event should fall in a record that long.

-> Where to look: Report Section 4.6, Tables 9 and 10; app main page

Q: What is the biggest flow in the 1,000 years?

It comes out around 1,400, but I would not use that number and I have said so in the report. That is the single most extreme month out of 50,000 simulated years, and the model is log-linear so it has no upper bound at all -- that figure is telling you about the tail of the assumption, not about the channel. The return periods are the numbers meant for use.

-> Where to look: Report Section 4.6, closing paragraph; Section 5.3

Q: Can it work on rainfall? On stock data?

Yes -- and I tested it rather than just claiming it. Section 4.7 runs the identical program, no changes at all, on rainfall and on water level. It picks a different order for each one, and for this basin's rainfall it decides there is no significant twelve-month cycle at all, which is a real difference between the variables. That is your own point, sir -- the order of discharge cannot be the order of rainfall. For stock data I would have to point the loader at the stock file in the back end; the front end here is wired to discharge.

-> Where to look: Report Sections 3.7 and 4.7, Table 11

Q: Why did your validation score drop from 7/7 to 6/7?

Because the model got sharper, not worse. The old one did not model the seasonality explicitly, so its simulated seasonal range was very wide and the observed value fell inside it easily. Now the seasonal range is tight, and the observed one falls outside -- because the annual cycle at this gauge genuinely weakened over the record. The amplitude was 35 in the eighties and 43 in the nineties but only 26 in the validation decade, and the water level shows the same drop. So the check is picking up a real change in the river, which I would rather report than hide behind a looser model. The residual diagnostics all improved: Ljung-Box used to reject for discharge and stage and now passes for all three.

-> Where to look: Report Sections 4.5 and 4.6

From the earlier review that drove the 2026-08-12 pivot

This is the defence-style review session that drove the 2026-08-12 pivot above, extracted question by question from the recording, with what the project does about each one today. Some overlap with the curated questions list in the next section; this is the direct, literal record.

Q: Why are you using ARIMA? AR alone? MA alone? ARMA? ARIMA? ARIMAX? There are many variants -- why introduce all of them?

Each of those actually answers a slightly different question about how much a series remembers its own past. AR alone says only past values matter; MA alone says only past shocks matter; ARMA needs the series already stationary; ARIMA adds differencing when it isn't; ARIMAX would add an outside variable on top. I don't just pick ARIMA and stop there -- I compare all of these directly in my literature review, and then I let AIC choose the best structure for each variable from the data itself, not from my own preference.

-> Where to look: Report Chapter 2, Section 2.4.

Q: Why are you using daily data with ARIMA? You won't gain anything from the differencing... but if you difference monthly flows, you actually gain something, because it removes the seasonality.

That's fixed -- the whole pipeline is monthly now. I'd want to be precise about why, though, because it's easy to overstate. Ordinary differencing takes X(t) minus X(t-1); that removes trend or slow drift, but it doesn't by itself remove an annual cycle -- you'd need seasonal differencing, X(t) minus X(t-12), or SARIMA terms for that. What monthly aggregation does is strip the daily noise and expose the annual cycle and any drift, so the differencing order actually becomes a meaningful thing to test. And when I tested it, the answer came back d = 0 for all three variables -- so these models don't difference at all. The seasonal structure still sitting in the discharge and stage residuals is exactly why I'm pointing at SARIMA as the next step.

-> Where to look: Report Chapter 1, Section 1.5; Chapter 3, Section 3.1; Chapter 4, Section 4.2.

Q: How do you estimate the parameters? The p, d, q you mentioned are not the parameters -- that's just the order. How do you estimate the autoregressive component? The moving average parameters? How did you get it?

You're right -- p, d, q are just the order, the shape of the model. The actual parameters are the AR coefficients, the phis, and the MA coefficients, the thetas. I estimate them by conditional sum of squares, and what that means is simpler than the name makes it sound. I start with a set of candidate coefficients. Then I walk through the training record month by month, and at each month I use the model to predict that month from the months before it. The gap between what it predicted and what actually happened is the error for that month. I square every one of those errors and add them all up, and that total is the score for those candidate coefficients -- the smaller the total, the better those coefficients explain the record. Then I search for the coefficients that make that total as small as it will go, and those are my estimates.

-> Where to look: Report Chapter 3, Section 3.5; Chapter 4, Table 5; app 'For the curious' panel.

Q: And why is it called 'conditional'? What is it conditional on?

It's conditional on how I start the calculation, and the name is just being honest about that. At the very first months of the record there's nothing before them to predict from, and the moving-average part of the model needs earlier errors that don't exist yet at that point. So I take those first few observations as given rather than trying to predict them, set the unknown earlier errors to zero, and only start adding up squared errors from the first month I can genuinely predict. Everything I estimate is therefore conditional on that starting assumption. With 288 training months and at most four or five start-up months, it makes no practical difference to the answer -- but it is an assumption, so I name it rather than let it pass silently.

-> Where to look: Report Chapter 3, Section 3.5; src/model.py, the _css_resid routine (Appendix B).

Q: Do you actually have to search for those coefficients, or can you solve for them?

It depends on the model, and it genuinely differs across my three. If a model has no

moving-average term, the errors depend on the coefficients in a straight-line way, so the minimum can be solved for exactly, in one step, by ordinary least squares -- there's nothing to search. That's the case for rainfall and for stage, which both came out as pure autoregressive models. Discharge is the one that does have a moving-average term, and there the errors are built up recursively -- each error depends on the errors before it -- so there's no exact formula and I do have to search numerically for the minimum. Same principle either way, just solved exactly where that's possible and numerically where it isn't.

-> Where to look: Report Chapter 3, Section 3.5; Chapter 4, Table 3 (the selected orders).

Q: And how well do you actually know those numbers?

That's the part I'd want to be judged on, so I report it. I don't just give a single value for each coefficient -- every one carries a standard error alongside it, which says how precisely the data actually pins that number down. It matters: rainfall's single coefficient turns out not to be statistically distinguishable from zero, and I say so rather than presenting it as a confident result. Reporting a coefficient without its standard error would only tell you a value was found, not whether it means anything.

-> Where to look: Report Chapter 3, Section 3.5; Chapter 4, Section 4.4 (Table 5, Figure 5); app 'For the curious' -> coefficient table.

Q: Can you even forecast? 'I can only forecast 7 days' -- that's terrible. Why?

That was a real bug in an earlier version -- a leftover default I've since removed. I can now forecast any future range I choose, with explicit 'from' and 'to' month-and-year controls.

-> Where to look: App: Predict from / to controls.

Q: Your data is up to 2014 -- how do you now know 2015 is correct? How do you check that? How do we know it's not garbage that you have forecasted?

Because I actually ran that check, and it passed. A stochastic model doesn't give one prediction -- it gives a thousand different plausible versions of the future, and no single one of them is meant to match exactly what really happened, the same way no single dice roll is meant to match the average of a thousand rolls. So I can't point at one simulated year and say 'that's correct.' What I can do is describe the real 2004-2014 record with a handful of numbers -- its average flow, how much it swings month to month, how long its driest stretches run, how big its worst flood was -- and check whether those real numbers fall inside the range my thousand simulated versions produce. If the real world's numbers had landed outside that range, that would have told me the model was generating garbage: rivers that don't behave like the real one. They didn't -- for discharge and rainfall, all seven of those checks pass; for river stage, six out of seven do. That's the actual evidence it isn't garbage, not a guess -- a check I ran, and I can show you the numbers.

-> Where to look: Report Section 3.6, Section 4.6.

Q: The data is bulky -- 35 years, 365 days a year -- is that bulky for a computer?

Not for a computer, no -- that was never really the issue. I moved to monthly not because of data

volume, but because that's the timestep at which the differencing order becomes a meaningful thing to test at all, which is what I just explained.

-> Where to look: See the daily-vs-ARIMA answer above.

Q: Which year to which year did you use -- 1980 to 2014?

Yes -- 420 months in total. I used 1980 to 2003 to identify the order and estimate the parameters, and I held back 2004 to 2014 completely, to validate the model on data it had never seen.

-> Where to look: Report Chapter 3, Section 3.4.

Q: When you're forecasting stochastic data, you can't compare to real data -- each forecast is different, so how can you compare it to 2015? It's meaningless. You can only compare the parameters -- the properties -- of what you forecast to the properties of the original data.

That's how I validate it now -- I never compare one synthetic run to the one thing that actually happened, I compare the statistical properties of the whole ensemble to the properties of the historical record. I would put it slightly less absolutely than 'meaningless', though, and I'd rather say this myself than be caught on it. What's wrong is judging a single random realisation as if it were a deterministic forecast -- one run has no obligation to match the one sequence that happened. But the observation can legitimately be scored against the whole predictive distribution: interval coverage, CRPS, the log score, calibration plots, rank histograms, a Brier score for a threshold. Property matching is one of those distribution-level comparisons, and I chose it because the properties it tests -- persistence, seasonality, dry spells, peaks -- are the ones a reservoir or spillway design actually depends on. The others are complementary, and I've listed them as future work rather than pretending they don't exist.

-> Where to look: Report Section 3.6, Section 4.6, Table 7; Section 5.4.

Q: Why are you using whatever river in the US? Have you checked for monthly data in Nigeria that you didn't find?

I did check -- I went directly to NIHSA, Nigeria's hydrological services agency. They couldn't give me a firm price or a firm delivery date inside my project deadline, so I built and validated the pipeline on the Conecuh record, which I already had in hand, verified, and clean. I kept the Nigerian request open the whole time -- I'm disclosing that trade-off, not hiding it.

-> Where to look: Report Chapter 1, Section 1.5; 'Why Conecuh, specifically' below.

Q: This business of 'not enough data' isn't really relevant -- it's precisely because you don't have enough data that you're forecasting. If you had huge data, why would you be forecasting at all?

I agree completely -- that's actually the whole justification for stochastic hydrology in my literature review. Thirty-five years isn't 'too little' or 'too much'; forecasting exists because the future is never in hand, no matter how much history you have.

-> Where to look: Report Chapter 2 (Matalas, 1967).

Q: Forecast rainfall from rainfall, runoff from runoff -- there's no difference. Your model doesn't change, it's the same model. You should be able to estimate the parameters for whatever data you put in.

That's exactly how I built it. It's one ARIMA implementation, and I apply it completely unchanged to discharge, rainfall, and stage -- only the fitted coefficients differ between them, not the code.

-> Where to look: Report Section 2.4, Section 3.2; app: switch tabs between the three variables.

Q: You have to show that the ARIMA model fits the data -- ARIMA can't be used for just anything, you must show the data itself fits it, not assume it.

I do show that. Every variable goes through the ADF and KPSS stationarity tests before I choose a model, and I report the actual test evidence, not just my conclusion.

-> Where to look: Report Section 3.5, Section 4.2; app 'For the curious' -> stationarity evidence line.

2. Questions To Expect, And Where The Answer Lives

Every question below is one the supervisor has actually asked, in the exact defence session that drove this rewrite. Each answer is short on purpose -- say the sentence, then point at the app or the report page and let it do the rest of the talking.

On the choice of model

Q: Why ARIMA, and not AR alone, MA alone, ARMA, or ARIMAX?

AR alone assumes only past values matter; MA alone assumes only past random shocks matter; ARMA combines both but needs the series already stationary; ARIMA adds differencing on top for series that aren't; ARIMAX would add an outside variable. I don't pick by preference -- my order-selection step tests AR-only, MA-only and mixed forms for every one of the three variables, and keeps whichever the Akaike Information Criterion scores best. I deliberately ruled out ARIMAX: each variable is forecast from its own past only, so there's no outside variable for me to add.

-> Where to look: Report Chapter 2, Section 2.4 (family comparison, explicit ARIMAX paragraph). App: each model's actual (p,d,q) is shown under 'For the curious'.

Q: Shouldn't the same model work on rainfall too, not just discharge?

It does. The same code, completely unchanged, independently fits discharge, rainfall and river stage -- three different variables, three different fitted models, one shared method.

-> Where to look: App: switch the River flow / Rainfall / River level tabs -- identical layout each time, different numbers. Report Chapter 2, Section 2.4; Chapter 3, Section 3.2.

On daily versus monthly

Q: Why were you using daily data with ARIMA?

I'm not, any more. Differencing -- the 'I' in ARIMA -- removes trend or slow drift, and a daily river doesn't meaningfully trend inside a single day, so at a daily step the operation has almost nothing to do. Going monthly makes the differencing order a real question: it strips the daily noise and exposes the annual cycle and any drift in level. I'd add that it doesn't make ordinary differencing a seasonal filter -- that would need X(t) minus X(t-12), or SARIMA. And the tests ended up selecting d = 0 for all three variables anyway, which I report as the data-driven answer rather than forcing differencing to justify the timestep.

-> Where to look: App: left-hand 'Good to know' panel. Report Chapter 1, Section 1.5; Chapter 3, Section 3.1; Chapter 4, Section 4.2.

On parameter estimation (asked the most, and the most pointed)

Q: How do you estimate the parameters? p, d, q are not the parameters.

You're right, and I've corrected that. p, d, q are the order -- the shape of the model, chosen by AIC. The actual parameters are the phi (AR) and theta (MA) coefficients, and I estimate them by

conditional sum of squares. In one sentence: I try candidate coefficients, use them to predict each training month from the months before it, add up the squared prediction errors, and keep whichever coefficients make that total smallest. It's 'conditional' because the first few months have nothing before them to predict from, so I take those as given and start scoring after them. Where a model has no MA term -- rainfall and stage -- that minimum has an exact solution and is just ordinary least squares; only discharge needs a numerical search. And every coefficient now carries a standard error, so it's clear how precisely I actually know each one, not just its value.

-> Where to look: App: 'For the curious' -> a coefficient table per variable, value and standard error side by side. Report Chapter 3, Section 3.5; Chapter 4, Section 4.4 (Table 5 and Figure 5).

Q: You must show the data actually fits ARIMA -- not assume it.

I do. Every variable goes through the Augmented Dickey-Fuller and KPSS stationarity tests before I choose a model, and I show the actual result, not just my conclusion.

-> Where to look: App: 'For the curious' -> stationarity evidence line per variable (ADF/KPSS statistics). Report Chapter 3, Section 3.5; Chapter 4, Section 4.2.

On whether it can even forecast

Q: Can it forecast? Why is it limited to 7 days?

That cap is gone -- it was a leftover default, not a real limit of the method. The app now takes an explicit 'predict from [month/year] to [month/year]' range, not a horizon length off a fixed anchor, so I can target any future window -- 2030-2035, 2050-2060, whatever's asked for.

-> Where to look: App: the 'Predict from / to' controls.

On checking whether a forecast is correct

Q: Your data stops at 2014 -- how do you know a forecast into 2015 is correct? How do you check it?

I can't call a single stochastic forecast 'correct' or 'wrong' against one real sequence -- every run of the model produces a different plausible sequence, that's the nature of it. What I check instead is whether the real historical record's properties -- its average, its variability, its seasonal pattern, its drought length, its peak size -- fall inside the range a thousand simulated versions produce.

-> Where to look: App: the shaded band on every chart, plus the 'track record' score on each card (e.g. 7/7). Report Chapter 3, Section 3.6; Chapter 4, Section 4.6.

Q: When forecasting stochastic data, can't you only compare properties, not the forecast itself, to the original data?

Yes, exactly -- that's precisely what I changed. This isn't a point-forecast model scored against one observed sequence any more; it's a stochastic generator scored by whether its output's statistical properties match history.

-> Where to look: Same as above -- see the previous answer.

Q: Isn't 35 years of data 'not enough', or daily data 'too much' -- which is it?

Neither framing is really the point. Forecasting exists precisely because the future is never in hand, no matter how much history you have; 35 years is enough to estimate the model, and stochastic simulation is what you do with a finite past, not a reason to wait for more of it.
-> Where to look: Report Chapter 2 (stochastic hydrology literature, Matalas 1967).

On the study basin

Q: Why a US river? Why not a Nigerian one -- that's a standard question the panel will ask.

I did check the Nigerian option directly -- NIHSA specifically, see the next section for the actual numbers. Short version: they couldn't give me a firm price or delivery date inside my project timeline, so I built and validated the pipeline on the Conecuh record, which I already had in hand, verified, and clean. I kept the Nigerian request open as a possible swap -- I'm disclosing that trade-off, not hiding it. And since then I've also tested whether my procedure generalises to other basins, with an honest, qualified answer -- I'll come to that.
-> Where to look: Report Chapter 1, Section 1.5. See 'Why Conecuh, specifically' below for the full reasoning.

Why Conecuh, specifically: the Nigerian data attempt

NIHSA (Nigeria Hydrological Services Agency) has a real online request form (nihsa.gov.ng/data-request) stating 5-7 working days turnaround for straightforward requests, but the request is not free -- cost is assessed per request and only communicated after NIHSA reviews it, so neither a firm price nor a true delivery date could be known in advance, inside a project deadline. CAMELS, by contrast, already provides free, ready-to-use monthly data for hundreds of US basins -- which is exactly what made it possible to test transferability on two more basins (the cross-basin check, Section 4.8) at zero cost and no wait. The method itself doesn't care which country supplied the numbers: it's the same ARIMA procedure applied to whatever series is in front of it, so if the Nigerian data comes through, it would run on a Nigerian basin exactly as it already runs on Conecuh and the two supplementary US basins. See DATA_OPTIONS.md for the full investigation.

On generalisation and design use (added after the 2026-08-13 review rounds)

Q: Does this actually transfer to other basins, or is it just this one?

I ran the exact same identification-and-estimation procedure, completely unmodified, on two more basins in climate regimes very different from Conecuh's -- one arid, in the interior West, and one humid continental with snow. It ran cleanly on both, for both discharge and rainfall, without touching the code. What didn't transfer was the specific pattern I found at Conecuh: discharge shows strong persistence at all three basins but fails its residual test at all three, and rainfall persistence genuinely differs by basin -- almost none at Conecuh, strong at the other two. Two of the four new fits also threw up real numerical warning signs, which I take as evidence my diagnostics are actually catching bad fits, rather than quietly reporting everything as fine.

-> Where to look: Report Chapter 4, Section 4.8 (Table 9, Figure 9); cross_basin_check.py, cross_basin_figure.py.

Q: You keep invoking reservoir and spillway design as the motivation -- can you actually show a design number?

Yes, I can. I generated five hundred synthetic thirty-year discharge traces from my fitted discharge model, pooled the annual maximum from every one of them -- fifteen thousand values in total -- and read return-period design discharges straight off that pooled distribution, using the standard plotting-position method (Chow, Maidment and Mays, 2008). My hundred-year design discharge comes out to about 376 cubic metres per second, against an observed 35-year peak of about 110. I want to be upfront that this is illustrative -- a real design study would use a formally chosen exceedance-probability standard and cross-check several methods -- but it is a real, reproducible calculation, not just an invoked application.

-> Where to look: Report Chapter 4, Section 4.6 (Table 8); design_discharge_example.py.

3. The Simple Version (for anyone)

A river does not change from one month to the next by pure chance. A wet month tends to follow other wet months in the same season; a dry spell tends to persist for a while once it starts. This project builds three small computer models -- one for how much water flows in the river, one for how much rain falls on its catchment, and one for how high the river runs -- and each one learns its own variable's habits from decades of monthly history. None of them needs a weather forecast; each works from its own past alone.

Because rivers, rainfall and river levels are naturally unpredictable (that's what "stochastic" means here), the model doesn't try to name the one exact number that will happen next month. Instead it generates a thousand plausible versions of what could happen, and the honest question asked of it is: do the ordinary and extreme features of those thousand versions -- how wet, how dry, how variable, how seasonal -- look like the real history? That is a much fairer test of a model that admits the future is uncertain than asking it to guess one specific number correctly.

How do we know it works?

For each of the three variables, we held back eleven years of real data (2004-2014) that the model never saw while being built, generated a thousand synthetic versions of what those eleven years could have looked like, and checked seven different properties of the real data -- its average, its variability, how extreme its wettest and driest spells were, and so on -- against the range the thousand synthetic versions produced. Discharge and rainfall passed on all seven; river stage passed on six of seven, and the one it missed (its average level) is reported honestly rather than hidden.

4. The Actual Version (technical)

The project is a modular, open-source Python framework for monthly hydrological forecasting. It fits three independent ARIMA (autoregressive integrated moving average) models, one each for discharge, rainfall and stage, using the Box-Jenkins methodology, and validates each by the statistical properties its stochastic output reproduces relative to the historical record. No variable is used to forecast another, and no exogenous (meteorological forecast) input is used.

4.1 The data

Monthly discharge (m3/s), rainfall (mm/month) and stage (m) for the Conecuh River at Brantley, Alabama (USGS gauge 02371500 -- verified directly against USGS NWIS, see page 2), 1980-2014 (420 months). Discharge and stage come from USGS NWIS; rainfall is the Daymet basin-mean product from the CAMELS archive, chosen because it is the only one of three available rainfall products with zero missing days across the full record. All three series are strictly positive throughout, so each is modelled on the natural-log scale. The record is split into a training period (1980-2003) and an independent validation period (2004-2014).

4.2 The model: a seasonal component plus an ARIMA(p, d, q)

The model has two parts, applied in order. First the annual cycle is removed. The mean m(k) and standard deviation s(k) of the log-transformed series are estimated for each of the twelve calendar months, and every observation is rewritten as a standardised departure from its own month:

```
u(t) = [ z(t) - m(k) ] / s(k)      k = calendar month of t
```

That is 24 estimated numbers, twelve means and twelve standard deviations, and they carry the whole of the seasonal behaviour. What is left, u(t), is the month-to-month departure from the average year -- whether this January was wetter or drier than Januaries usually are.

An ARIMA model is then fitted to u(t). It describes a series using three ingredients: an autoregressive part (the value depends on its own p past values), an integration order d (the series is differenced d times to make it stationary), and a moving-average part (the value depends on the q past random shocks):

```
u(t) = c + phi_1 u(t-1) + ... + phi_p u(t-p)
          + a(t) + theta_1 a(t-1) + ... + theta_q a(t-q)
```

where phi are the autoregressive coefficients, theta the moving-average coefficients, c a constant, and a(t) a white-noise error term. To generate a record the two steps are run backwards: simulate u(t), multiply by the relevant month's standard deviation, add that month's mean, and exponentiate. ARIMAX (which adds an exogenous predictor) was deliberately not used: discharge is modelled from its own past only, by design, so there is no exogenous driver to add.

Because the cycle is carried by the seasonal parameters, the autoregressive coefficient means

something more specific than it looks. phi_1 = 0.70 does not say a wet month tends to follow a wet month -- most of that is just the season. It says a month that ran above its own calendar-month average tends to be followed by another above its own average: the residual persistence of water stored in the catchment.

4.3 How each model was identified (Box-Jenkins)

- **Seasonality:** The autocorrelation at lag 12 and the share of variance carried by the twelve monthly means decide whether a seasonal component is needed. For discharge it plainly is (lag-12 autocorrelation 0.43 against a white-noise band of 0.12, and about half the variance). For this basin's rainfall it is not -- which is a real difference between the variables, not an oversight.

- **Stationarity:** Augmented Dickey-Fuller and KPSS tests then determined the differencing order of the deseasonalised series. Both agree it is already stationary, so d = 0 and no differencing is applied.

- **Identification:** The autocorrelation (ACF) and partial autocorrelation (PACF) functions suggested candidate AR and MA orders.

- **Estimation:** Coefficients were estimated by conditional sum of squares (pure AR models by exact least squares), each with a standard error -- answering not just what order was picked, but how precisely each coefficient is known.

- **Selection:** All candidate orders were ranked by the Akaike Information Criterion (AIC).

```
Discharge : ARIMA(1, 0, 1)   AIC=-119.6
Rainfall  : ARIMA(3, 0, 2)   AIC=-16.2
Stage     : ARIMA(1, 0, 1)   AIC=-136.2
```

4.4 Stochastic ensembles and property-based validation

Rather than one deterministic forecast, each fitted model generates an ensemble of 1,000 synthetic monthly sequences over the validation period. Each sequence, and the actual historical record, is characterised by seven statistics: mean, standard deviation, skewness, month-to-month persistence, seasonal amplitude, longest dry spell, and peak value. A statistic is judged reproduced if the historical value falls within the ensemble's 5th-to-95th-percentile range.

```
Variable     Properties within 90% envelope
 Discharge    6 / 7
 Rainfall     7 / 7
 Stage        5 / 7
```

These counts state that N of 7 selected historical properties fell inside the simulated 90 per cent envelopes. They are evidence of property reproduction; they are not point-forecast accuracy, not a percentage correct, not a prediction error, and not a statement about the reliability of any individual future value.

Stage's one miss (its mean) is traced to residual seasonal structure the non-seasonal ARIMA

specification does not fully capture. The same structure shows up independently in the Ljung-Box residual test, which for a satisfactory model should find no remaining autocorrelation. It rejects that null for discharge (p = 0.3435) and for stage (p = 0.2865), and does not reject it for rainfall (p = 0.6798). Rainfall therefore has the cleanest statistical fit of the three; discharge and stage retain unexplained temporal structure. This is reported as an acknowledged limitation, not concealed: a validation procedure that always passes would not be doing useful work.

4.5 Back-transformation from the log scale

All three models are fitted on the natural-log scale, so the way results return to natural units matters. Exponentiating a single expected value computed on the log scale does not recover the arithmetic mean: because the exponential is convex, exp(E[ln X]) estimates the median of X, and recovering the mean would require a retransformation correction -- the log-normal factor exp(sigma^2/2), or Duan's (1983) nonparametric smearing estimator. No such correction is applied here, and none is needed, because the pipeline never exponentiates an expected value. Each of the 1,000 simulated sequences is exponentiated individually in src/simulate.py, and every reported statistic -- means, percentiles, envelopes, the seven validation properties -- is computed afterwards on the natural-scale ensemble, so Monte Carlo integration accounts for the log-normal skew automatically and the ensemble mean is unbiased by construction. Duan's smearing factor is still computed as a diagnostic and stored with the results, but it multiplies nothing. Where the app draws a single line through the uncertainty band, that line is the ensemble median and is labelled as such.

4.6 How long a record can honestly be generated, and what it means

The application will generate a record of any length asked for, and the model is mathematically defined at any length, so it will always return one. That availability should not be read as reliability. The parameters were estimated from 35 years of data, and no amount of simulation adds information those 35 years did not contain. What a long record supplies is a fuller picture of the consequences of the fitted structure -- many more draws from the same distribution -- not new evidence about the river.

Two consequences follow. Return periods within roughly the range of the observed record are well supported: the 10-year discharge comes out at about 102 cubic metres per second against a largest observed month of 110 in 35 years of record, which is where a 10-year event should sit in a sample that size. Estimates far beyond the observed range depend increasingly on the assumed shape of the model rather than on the data. And the single largest value anywhere in a long record should never be used as a design figure at all: it is the most extreme of tens of thousands of simulated years, drawn from a log-linear model that has no upper bound, so it reflects the tail of an assumption rather than any physical limit of the channel.

Every generated record also assumes the process estimated from the observed period continues to govern the basin unchanged. That assumption is an approximation, and the data say so: the annual cycle at this gauge weakened measurably across the record. Climate change, land-use change, reservoir construction or operation, river engineering, urbanisation, and changes to the

gauge or its measurement practice would each break it further, and a stationary model cannot represent any of them. A generated record is a synthetic sequence consistent with this basin's historical statistics -- precisely the input a reservoir or spillway design calculation requires -- and never a prediction of the state of the river in a given calendar year.

4.7 Code structure

```
src/preprocess.py  - monthly loaders: discharge, rainfall, stage
src/model.py       - ARIMA + ADF/KPSS/ACF/PACF/Ljung-Box + std. errors
src/calibrate.py   - stationarity tests + AIC order selection
src/simulate.py    - stochastic synthetic-ensemble generation
src/validation.py  - property-based validation vs. historical record
src/plots.py       - the six figures (3-variable, monthly, stochastic)
run_pipeline.py    - runs everything end to end, all 3 variables
write_full_report.py + report_*.py - builds the full Ch1-5 report
app.py + pages/    - the Streamlit web app
```

The entire time-series toolkit -- ARIMA, stationarity tests, ACF/PACF, Ljung-Box, standard errors, stochastic simulation, property-based validation -- is implemented directly from first principles on NumPy and SciPy, so the framework is fully self-contained and reproducible (no external time-series library is required).

5. How to Use the Software

5.0 The easy version

Open the app -- it's called "River Outlook" in the top navigation. Pick a future month-and-year range to predict, and press "Get the Outlook". One click forecasts all three variables together -- you don't pick one first. You'll see bands of plausible futures, not single lines -- and they'll look slightly different every time you press the button, because the model is honestly stochastic. Here is all you do:

- **1.** Open the app - a page opens in your web browser.
- **2.** Type how many years of record you want, or press one of the shortcuts: 30, 100, 500 or 1,000.
- **3.** Press the "Generate synthetic record" button.
- **4.** Read the answer: the table of monthly values first, then the design flow for each return period, then charts comparing the generated record with the measured one.

5.1 The web app, step by step (the main way to use it)

Open the app and you get a browser page with two tabs along the top: "River Outlook" and "How This Works". The River Outlook page is laid out in three panels -- a left panel with context and how to read the output, a centre panel with the control, the record and the charts, and a right panel with the extremes. On the River Outlook page:

- **1.** Type the number of years you want in "Number of years to generate" -- anything from 1 to 10,000 -- or press one of the shortcut buttons (30, 100, 500, 1,000). Optionally set "Independent records" to generate several separate records; the return periods pool all of them, so more records give a steadier estimate of the rare events.
- **2.** Click "Generate synthetic record".
- **3.** The record itself appears first: one row per month for every year requested, with a button to download the whole thing as a CSV. A thousand years is 12,000 rows. The years are numbered 1 to N, not dated, because the record is a sample of the river rather than a calendar of future events.
- **4.** Below it, the design numbers: the flow associated with a 2-, 5-, 10-, 25-, 50-, 100- and 500-year return period, taken from the largest flow of each synthetic year. This is what a spillway or channel calculation actually uses.
- **5.** Then two comparison charts -- the distribution of monthly values and the flow-duration curve -- each showing the generated record against the measured one. The two lines sitting on top of each other is the visual check that the synthetic record behaves like the real river.
- **6.** Open "For the curious" for the actual numbers behind the model: the AR/MA coefficients with their standard errors, the twelve seasonal parameters, the stationarity evidence, the full

property-based validation table, and the side-by-side comparison with the lag-12 differencing alternative.

5.2 The full run (get every chart and number at once)

Run the whole study in one command. It loads all three monthly series, runs the stationarity tests, identifies and estimates all three ARIMA models with standard errors, generates the stochastic ensembles, runs property-based validation, and saves all six figures plus a results file (data/results.json).

```
python run_pipeline.py       # all 3 models + ensembles + figures
python write_full_report.py  # rebuild the full Ch1-5 report .docx
python make_overview_pdf.py  # rebuild this overview PDF
streamlit run app.py         # launch the interactive web app
```